

Reviewer Pack — Cheeger Constant Exponentiates Resolution Length for Tseitin...

Entry 3fd17de5d8ec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 21:52:57 UTC

Cheeger Constant Exponentiates Resolution Length for Tseitin Formulas

Entry ID: 3fd17de5d8ec **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 21:52:57 UTC

1. Conjecture

Field A (mathematical branch): Spectral Graph Theory **Field B** (complexity object): Resolution Refutation Length

Statement:

For any connected graph G with n vertices, the Resolution refutation length of the Tseitin formula $\tau(G)$ is $\geq 2^{\Omega(h(G))}$, where $h(G)$ is the Cheeger constant of G . For graphs with $h(G) \leq 1/\sqrt{n}$, this bound is $\leq 2^{O(1)}$.

Rationale (proposer's reasoning):

Cheeger constants capture expansion properties critical for Tseitin formula hardness. Expanders ($h(G) \approx 1$) force exponential refutation lengths via the cumulative entropy framework, while non-expanders ($h(G) \approx 0$) admit polynomial-length proofs due to low expansion.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dfffae72b02b66c6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Cheeger constant resolution refutation length Tseitin formulas - spectral graph theory Cheeger constant resolution complexity Tseitin formulas - Cheeger constant resolution lower bounds Tseitin formulas $h(G) \leq 1/\sqrt{n}$

Top relevant hits considered: - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/1807.02225v2] Cheeger inequalities for graph limits - [http://arxiv.org/abs/2404.04038v1] Refutability as Recursive as Provability - [http://arxiv.org/abs/2103.14334v4] Invariant subspaces of elliptic systems II: spectral theory - [http://arxiv.org/abs/1411.3530v4] Cheeger constants, structural balance, and spectral clustering analysis for signed graphs - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^\pm \rightarrow D(\rightarrow K_S^0 h'^+ h'^-) h^\pm$ dec - [http://arxiv.org/abs/2604.05712v1] Precise measurement of the CKM angle γ with a novel approach - [http://arxiv.org/abs/2410.02421v2] Search for lepton number violating decays of $D_s^+ \rightarrow h^- h^0 e^+ e^+$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_regular_graph(n, k):
    if 2 * k > n - 1:
        raise ValueError("Invalid parameters for regular graph generation")

    adj = [[] for _ in range(n)]
    degree_sum = 0

    for i in range(n):
        neighbors = random.sample(range(i + 1, min(i + k // 2 + 1, n)), k // 2)
        adj[i] = neighbors
        degree_sum += len(neighbors)

    if degree_sum % 2 != 0:
        raise ValueError("Failed to generate a regular graph")

    return adj

def spectral_cheeger_constant(adj):
    n = len(adj)
    laplacian = [[0] * n for _ in range(n)]

    for i in range(n):
```

```

deg_i = len(adj[i])
laplacian[i][i] = deg_i
for j in adj[i]:
    laplacian[i][j] = -1

# Normalize the Laplacian
for i in range(n):
    sum_row = sum(laplacian[i])
    for j in range(n):
        laplacian[i][j] /= math.sqrt(sum_row)

# Compute eigenvalues of the normalized Laplacian
eigenvalues = [0] * n
for _ in range(10): # Power iteration method
    v = [random.random() for _ in range(n)]
    v = [x / sum(v) for x in v]
    v_next = [sum(laplacian[i][j] * v[j] for j in range(n)) for i in range(n)]
    v_next = [x / sum(v_next) for x in v_next]
    eigenvalues[0] += max(abs(x - y) for x, y in zip(v, v_next))
    v = v_next

return min(eigenvalues)

def tseitin_formula(adj):
    n = len(adj)
    literals = [f"x{i}" for i in range(n)]
    clauses = []

    for i in range(n):
        clause = [literals[i]]
        for j in adj[i]:
            clause.append(f"~{literals[j]}")
        clauses.append(clause)

    return clauses

def dpll_solver(clauses, assignment):
    if not clauses:
        return True
    unit_clause = next((c for c in clauses if len(c) == 1), None)
    if unit_clause:
        literal = unit_clause[0]
        value = literal.startswith("~")
        literal = literal.lstrip("~")
        if literal in assignment and assignment[literal] != value:
            return False
        assignment[literal] = value
        clauses = [c for c in clauses if literal not in c and not any(l.startswith("~") and l[1:]
        pure_literal = next((l for l, count in Counter([x.lstrip("~") for x in sum(clauses, [])]).items() if count == 1), None)
        if pure_literal:
            value = pure_literal.startswith("~")
            literal = pure_literal.lstrip("~")
            assignment[literal] = value
            clauses = [c for c in clauses if literal not in c and not any(l.startswith("~") and l[1:]

literals = list(assignment.keys())

```

```

literal = random.choice(literals)
value = assignment[literal]
new_assignment = {k: v for k, v in assignment.items()}
new_assignment[literal] = not value
if dpll_solver(clauses, new_assignment):
    return True

new_assignment[literal] = value
clauses.append([f"~{literal}"])
return dpll_solver(clauses, new_assignment)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    k = 8
    adj = generate_random_regular_graph(n, k)
    h_G = spectral_cheeger_constant(adj)

    clauses = tseitin_formula(adj)
    assignment = {}

    proof_steps = 0
    while not dpll_solver(clauses, assignment):
        proof_steps += 1

    metric_value = proof_steps
    conjecture_holds = metric_value >= 2 ** (h_G * math.log(n))
    counterexample = "" if conjecture_holds else f"Graph with h(G)={h_G} and proof steps={proof_steps}"

    return {
        "metric_name": "Proof Steps",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")

```

```

else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Graph with  $h(G) \leq 1/\sqrt{n}$ \" first_failing_seed={fi

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9c27fc76.py", line 140, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9c27fc76.py", line 112, in run_trial
    adj = generate_random_regular_graph(n, k)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9c27fc76.py", line 26, in generate_rand
    neighbors = random.sample(range(i + 1, min(i + k // 2 + 1, n)), k // 2)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 430, in sample
    raise ValueError("Sample larger than population or is negative")
ValueError: Sample larger than population or is negative

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to invalid sampling; no data produced to evaluate conjecture | next: Fix generate_random_regular_graph to handle $k/2 \leq n-i-1$ for all i

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112140
2	propose	ollama_remote	qwen3:8b	0	0	89705
3	preregistration	ollama_remote	qwen3:8b	0	0	24098
4	novelty	ollama_remote	qwen3:8b	0	0	20163
5	novelty	ollama_remote	qwen3:8b	0	0	24742
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18510
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15020
8	judge	ollama_remote	qwen3:8b	0	0	16750

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 321127 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/3fd17de5d8ec.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3fd17de5d8ec.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3fd17de5d8ec.tar.gz (if generated)